

S-BiAL: Sparse Bilevel Augmented Lagrangian for Unlearning

1 Formulation

We propose **S-BiAL**, a *Sparse Bilevel Augmented Lagrangian* framework for machine unlearning. The method integrates three complementary ideas: (i) a *bilevel structure* that explicitly separates retain and forget dynamics; (ii) a *constraint controller* in the standard dual form used in the unlearning literature; and (iii) *sparsity* via a regularizer $R(\cdot)$ inside the inner level to improve stability and efficiency [2, 1, 3]. We first present the base constrained bilevel formulation, then extend it with a fixed binary mask and an Augmented-Lagrangian solver.

1.1 Base Bilevel Constrained Unlearning

Let $\mathcal{D} = \mathcal{D}_{\text{retain}} \cup \mathcal{D}_{\text{forget}}$ and $\theta \in \mathbb{R}^d$ denote model parameters. Define

$$L_{\text{ret}}(\theta) = \frac{1}{|\mathcal{D}_{\text{retain}}|} \sum_{(x,y) \in \mathcal{D}_{\text{retain}}} \ell(\theta; x, y), \quad L_{\text{fgt}}(\theta) = \frac{1}{|\mathcal{D}_{\text{forget}}|} \sum_{x \in \mathcal{D}_{\text{forget}}} \Phi(z_{\theta}(x)), \quad (1)$$

where $z_{\theta}(x)$ are logits. We adopt the *logit-margin flattening* forget loss

$$\Phi(z) = \max_j z_j - \frac{1}{C} \sum_{j=1}^C z_j, \quad (2)$$

which encourages uniform softmax by driving logits toward equality. The allowed utility degradation is $\varepsilon = (1 + \alpha) L_{\text{ret}}(\theta_{\text{ref}})$ for a small $\alpha > 0$.

Outer problem (standard dual form). The constrained unlearning objective is written as

$$\min_{\lambda \geq 0} \mathcal{L}(\theta^*(\lambda), \lambda) := L_{\text{fgt}}(\theta^*(\lambda)) + \lambda(L_{\text{ret}}(\theta^*(\lambda)) - \varepsilon), \quad (3)$$

where $\theta^*(\lambda)$ is the inner solution defined below.

Inner problem (retain + sparsity). For each λ , the retained model is obtained by

$$\theta^*(\lambda) \in \arg \min_{\theta} L_{\text{ret}}(\theta) + \gamma R(\theta), \quad (4)$$

with $\gamma > 0$ and $R(\theta)$ a sparsifying regularizer (e.g., $\|\theta\|_1$ or group $\ell_{2,1}$). Here, L_{ret} measures utility on the retained data, L_{fgt} drives selective forgetting on the forget set, ε sets the explicit utility budget, λ enforces the constraint in the usual dual manner, and $\gamma R(\theta)$ induces sparsity to stabilize the inner solution.

Why S-BiAL is a Bilevel Problem. Although the dual formulation in (3) could be viewed as a single constrained program, we explicitly maintain a *bilevel structure* because two distinct learning objectives—*retain* and *forget*—play fundamentally different roles. The inner level models the process of preserving useful knowledge: it retrains or refines the parameters θ on the retained data under a sparsity regularizer $\gamma R(\theta)$ to obtain a stable reference model θ^* . The outer level then decides how to adjust this model to enforce forgetting behavior while respecting a prescribed retention budget. Keeping the two levels separate clarifies both optimization and interpretation: the inner loop captures “how the model would behave if trained only for retention,” while the outer loop measures and regulates “how much that behavior may change when enforcing forgetting.” This separation also allows the use of implicit differentiation—a standard bilevel technique—to propagate gradients from the outer forget objective through the implicitly defined inner optimum θ^* .

In BLUR [3], the hierarchy is inverted: the *inner* problem minimizes the forget loss, and the *outer* problem recovers utility on the retain set. That design emphasizes safety—guaranteeing complete forgetting first, then restoring accuracy as a secondary task. In contrast, S-BiAL adopts a *retention-first* view motivated by practical deployment. Our inner problem stabilizes the model on retained data and imposes sparsity to reduce variance and computational load; the outer Augmented-Lagrangian controller then applies forgetting operations subject to an explicit utility constraint $L_{\text{ret}}(\theta) \leq \varepsilon$. This reversal makes S-BiAL more suitable for scenarios where maintaining baseline performance is essential and forgetting must proceed in a controlled, quantifiable manner. Both formulations are bilevel in structure, but they encode different priorities: BLUR enforces “forget completely, then recover,” whereas S-BiAL enforces “retain reliably, then forget safely.”

2 Mask-Based Sparsity and Augmented-Lagrangian Solver

While $\gamma R(\theta)$ promotes sparsity, we further *instantiate* sparsity via a fixed binary mask to obtain a lower-dimensional, better-conditioned inner landscape. Let $m \in \{0, 1\}^d$ be a mask (chosen once from θ_{ref} by magnitude, SynFlow, or OMP) and parameterize

$$\theta = m \odot w. \quad (5)$$

The regularizer is applied on the active set:

$$R(\theta) = \sum_{i: m_i=1} r(\theta_i), \quad \text{e.g., } r(\theta_i) = |\theta_i| \text{ or } \|\theta_G\|_2 \text{ for group } G. \quad (6)$$

Substituting (5) into (4)–(3) yields the **S-BiAL** instantiation:

$$\textbf{Inner (retain + masked sparsity): } \theta^*(\lambda) \in \arg \min_{\theta} L_{\text{ret}}(\theta) + \gamma R(\theta), \quad \text{s.t. } \theta = m \odot w, \quad (7)$$

$$\textbf{Outer (forget + constraint): } \min_{\lambda \geq 0} \mathcal{L}(\theta^*(\lambda), \lambda) := L_{\text{fgt}}(\theta^*(\lambda)) + \lambda(L_{\text{ret}}(\theta^*(\lambda)) - \varepsilon). \quad (8)$$

This *constrained bilevel* structure is consistent with prior unlearning formulations such as BLUR [3], but reverses the hierarchy: the inner level ensures stable retention under sparsity, while the outer level drives forgetting subject to a fixed utility budget ε .

Implicit-gradient correction. Differentiating through the inner argmin gives the standard implicit correction to the outer gradient:

$$\nabla_{\theta} \mathcal{L}(\theta, \lambda) = \underbrace{\nabla_{\theta\theta}^2 \mathcal{L}(\theta, \lambda)}_{\text{HVP only}} \cdot \underbrace{\left(\nabla_{\theta\theta}^2 (L_{\text{ret}}(\theta) + \gamma R(\theta)) \right)^{-1}}_H \cdot \underbrace{\nabla_{\theta} L_{\text{ret}}(\theta)}_{\text{inner KKT}}, \quad (9)$$

which we approximate using Hessian–vector products (CG or truncated Neumann) *on the active coordinates* $\text{supp}(m)$, improving conditioning and efficiency. *In practice*, we instantiate $\mathcal{L}$ as the Augmented–Lagrangian objective $\mathcal{L}_{\rho}$ defined in §2.1.

2.1 Why an Augmented–Lagrangian Bilevel Solver and Implicit Hypergradients

Why Augmented–Lagrangian (ALM)? Our constrained outer problem (retain budget) is nonconvex and coupled to an inner learner. A pure dual-ascent update on the multiplier λ can oscillate: the retain residual $L_{\text{ret}}(\theta) - \varepsilon$ may change sign as θ moves, especially under minibatching. ALM stabilizes the residual by adding a quadratic penalty

$$\frac{\rho}{2} (L_{\text{ret}}(\theta) - \varepsilon)^2, \quad \rho > 0,$$

so the outer step optimizes a smoothed landscape. Intuitively, λ provides direction (*push down residual*), while ρ provides curvature (*damp oscillations*). We increase ρ only when the residual stalls, avoiding an overly stiff problem.

Why implicit hypergradients? The outer step should respect that θ is the solution (or near-solution) of the *inner* problem

$$\theta^* \in \arg \min_{\theta} L_{\text{ret}}(\theta) + \gamma R(\theta),$$

thus naively differentiating the outer objective $\mathcal{L}_{\rho}(\theta, \lambda)$ as if θ were free ignores the inner optimality structure and can yield biased or high-variance updates. Implicit differentiation encodes the fact that θ changes *as an inner minimizer* when we perturb the outer objective: it corrects the outer gradient by removing components that would be “undone” by a precise re-solve of the inner problem.

Derivation (fixed-point / KKT view). Let $F(\theta) := \nabla_{\theta} (L_{\text{ret}}(\theta) + \gamma R(\theta))$. At an exact inner solution, $F(\theta^*) = 0$. Differentiate $F(\theta^*) = 0$ w.r.t. an outer perturbation that changes $\mathcal{L}_{\rho}(\theta, \lambda)$ (holding data fixed):

$$\nabla_{\theta\theta}^2 (L_{\text{ret}} + \gamma R) \frac{d\theta^*}{d(\text{outer})} + \nabla_{\theta} (\text{outer influence on inner}) = 0.$$

When the inner objective has no explicit outer term (as here), the second term is zero and we get the classic

$$\frac{d\theta^*}{d(\text{outer})} = -H^{-1} \underbrace{0}_{\text{explicit term}} = 0,$$

but the outer objective still depends *directly* on θ^* ; to compute a descent direction for the outer we must remove the component that would be neutralized by a re-minimization of the inner objective.

This yields the standard *implicit correction* to the outer gradient:

$$g_{\text{outer-corr}} = \nabla_{\theta} \mathcal{L}_{\rho}(\theta, \lambda) - \nabla_{\theta\theta}^2 \mathcal{L}_{\rho}(\theta, \lambda) H^{-1} \nabla_{\theta} L_{\text{ret}}(\theta), \quad (10)$$

where $H := \nabla_{\theta\theta}^2 (L_{\text{ret}}(\theta) + \gamma R(\theta))$. The first term is the naive outer gradient; the second removes the component along directions that the inner solver would immediately neutralize. This improves stability and reduces variance of the outer updates.

How we compute it efficiently. We never form H or H^{-1} . Instead, we solve $H h = v$ for h using *conjugate gradients* (CG) with Hessian–vector products (HVPs); here $v = \nabla_{\theta} \mathcal{L}_{\rho}(\theta, \lambda)$. In practice:

1. Compute $v \leftarrow \nabla_{\theta} \mathcal{L}_{\rho}(\theta, \lambda)$ on the current minibatches.
2. Approximately solve $H h = v$ via CG (5–10 iterations suffice).
3. Form $g_{\text{outer-corr}} = v - (\nabla_{\theta\theta}^2 \mathcal{L}_{\rho}(\theta, \lambda)) h$.

All second-order terms use HVPs via autodiff; no dense matrices are stored.

Why masking helps. We apply all inner and outer updates *only* on the active coordinates $\text{supp}(m)$ and compute all HVPs in this subspace. This reduces dimensionality and improves H conditioning, so CG converges in a handful of iterations, making implicit gradients practical.

When to skip implicit correction. If sparsity is high (e.g., 90–95%) or models are small, the naive outer gradient often works well. We recommend reporting both variants (with/without correction) as an ablation; the corrected variant usually converges in fewer outer steps and exhibits lower variance.

Summary. ALM stabilizes the *constraint*; implicit differentiation stabilizes the *bilevel coupling*. Masking makes both fast and reliable in practice.

Algorithm

2.2 Practical Defaults, Complexity, and Troubleshooting

Default hyperparameters (safe starting points).

Mask sparsity	80–95% (higher for larger models)
Retain budget	$\alpha \in \{0.01, 0.02, 0.05\}$ so $\varepsilon = (1 + \alpha) L_{\text{ret}}(\theta_{\text{ref}})$
Regularizer	$R(\theta) = \ \theta\ _1$ on active coords; $\gamma \in [10^{-6}, 10^{-3}]$
AL penalty	$\rho \in [0.1, 1.0]$; double if $ r $ stalls for 3–5 outer steps
Stepsizes	Inner η_{in} as in standard finetuning; Outer $\eta_{\theta} \in [0.1, 1.0] \times \eta_{\text{in}}$
Inner steps	$K = 10$ –50 per outer iteration (short, warm-started)
Implicit solve	CG iters 5–10 with tol 10^{-3} (on active coords only)
Batching	Use <i>separate</i> mini-batches for retain vs. forget each outer step

Algorithm 1 S-BiAL: Augmented–Lagrangian solver with implicit outer correction (masked)

Inputs: Pretrained θ_0 ; retain/forget datasets $\mathcal{D}_{\text{retain}}, \mathcal{D}_{\text{forget}}$; mask m ; budget ε ; regularizer R with weight γ ; ALM penalty ρ ; stepsizes $\eta_{\text{in}}, \eta_{\theta}$; inner steps K .

Outputs: Unlearned sparse parameters $\hat{\theta}$.

```
1: Initialize  $\theta \leftarrow m \odot \theta_0, \quad \lambda \leftarrow 0$ .
2: for  $t = 0, 1, \dots, T - 1$  do
3:   Inner solve (retain +  $\gamma R$  on active set).
4:   for  $k = 1$  to  $K$  do
5:     Sample retain minibatch  $\mathcal{B}_{\text{ret}}$ .
6:      $g_{\text{in}} \leftarrow \nabla_{\theta}[L_{\text{ret}}(\theta; \mathcal{B}_{\text{ret}}) + \gamma R(\theta)]$ .
7:      $\theta \leftarrow \theta - \eta_{\text{in}}(m \odot g_{\text{in}})$  ▷ updates only on  $\text{supp}(m)$ 
8:   end for
9:   Outer objective pieces (separate batches).
10:  Sample forget minibatch  $\mathcal{B}_{\text{fgt}}$  and a fresh retain minibatch  $\mathcal{B}'_{\text{ret}}$ .
11:   $r \leftarrow L_{\text{ret}}(\theta; \mathcal{B}'_{\text{ret}}) - \varepsilon$  ▷ constraint residual
12:   $g_{\text{raw}} \leftarrow \nabla_{\theta}[L_{\text{fgt}}(\theta; \mathcal{B}_{\text{fgt}}) + \lambda L_{\text{ret}}(\theta; \mathcal{B}'_{\text{ret}})]$ .
13:   $g_{\text{alm}} \leftarrow g_{\text{raw}} + \rho r \nabla_{\theta} L_{\text{ret}}(\theta; \mathcal{B}'_{\text{ret}})$ .
14:  (Optional) Implicit correction (HVP + CG on active coords).
15:     $v \leftarrow m \odot g_{\text{alm}}$ .
16:     $H(\cdot) \approx \nabla_{\theta\theta}^2(L_{\text{ret}}(\theta; \mathcal{B}'_{\text{ret}}) + \gamma R(\theta))[\cdot]$  ▷ HVP oracle
17:    Solve  $Hh = v$  by CG (5–10 iters) to tolerance  $10^{-3}$  on  $\text{supp}(m)$ .
18:     $g_{\text{outer}} \leftarrow v - (\nabla_{\theta\theta}^2 \mathcal{L}_{\rho}(\theta, \lambda))[h]$  ▷ HVP again
19:    Primal masked outer step:  $\theta \leftarrow \theta - \eta_{\theta} g_{\text{outer}}$ .
20:    Dual update (ALM):  $\lambda \leftarrow \max\{0, \lambda + \rho r\}$ .
21:  end for
22: return  $\hat{\theta} \leftarrow \theta$ .
```

Compute/memory profile. Compared to ordinary finetuning, the extra cost comes from (i) the outer backward pass through L_{fgt} and the AL terms, and (ii) 5–10 Hessian–vector products for the CG solve on the *active* coordinates. With 80–95% sparsity, this overhead is typically modest and *sublinear* in full model size.

Common symptoms and fixes.

- **Oscillating retain loss:** increase ρ ; consider smaller outer step η_{θ} ; ensure retain vs. forget batches are different.
- **No forgetting progress:** increase outer steps or η_{θ} ; slightly reduce α (tighter budget loosens forgetting headroom).
- **Unstable CG/implicit step:** clip g_{alm} ; lower CG tol or iters; ensure R is applied only on active coords; check mask covers each layer sufficiently.
- **Retain drops below reference:** lower ρ or cap λ to avoid over-penalizing; verify ε computed from the correct θ_{ref} .

Ablations (recommended for a student implementer).

1. **No implicit correction:** drop the CG step and report the gap (often small at high sparsity).
2. **Dual-ascent only:** set $\rho=0$ and show higher variance/oscillation.
3. **L1-adaptive (no mask):** replace mask with plain ℓ_1 ; expect slower convergence and noisier hypergradients.

References

- [1] Taha Entesari, Arman Hatami, Rinat Khaziev, Anil Ramakrishna, and Mahyar Fazlyab. Constrained entropic unlearning: A primal–dual framework for large language models. *arXiv preprint arXiv:2506.05314*, 2025.
- [2] Jinghan Jia, Jiancheng Liu, Parikshit Ram, Yuguang Yao, Gaowen Liu, Yang Liu, Pranay Sharma, and Sijia Liu. Model sparsity can simplify machine unlearning. In *Advances in Neural Information Processing Systems (NeurIPS 36)*, 2023.
- [3] Hadi Reisizadeh, Jinghan Jia, Zhiqi Bu, Bhanukiran Vinzamuri, Anil Ramakrishna, Kai-Wei Chang, Volkan Cevher, Sijia Liu, and Mingyi Hong. Blur: A bi-level optimization approach for llm unlearning. *arXiv preprint arXiv:2506.08164*, 2025.